

Example: IQ scores

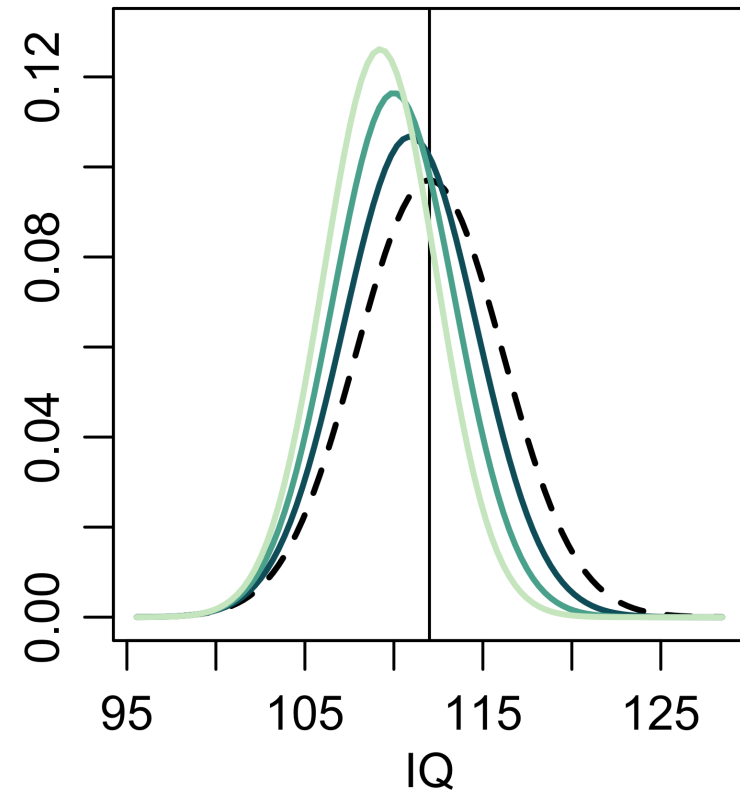
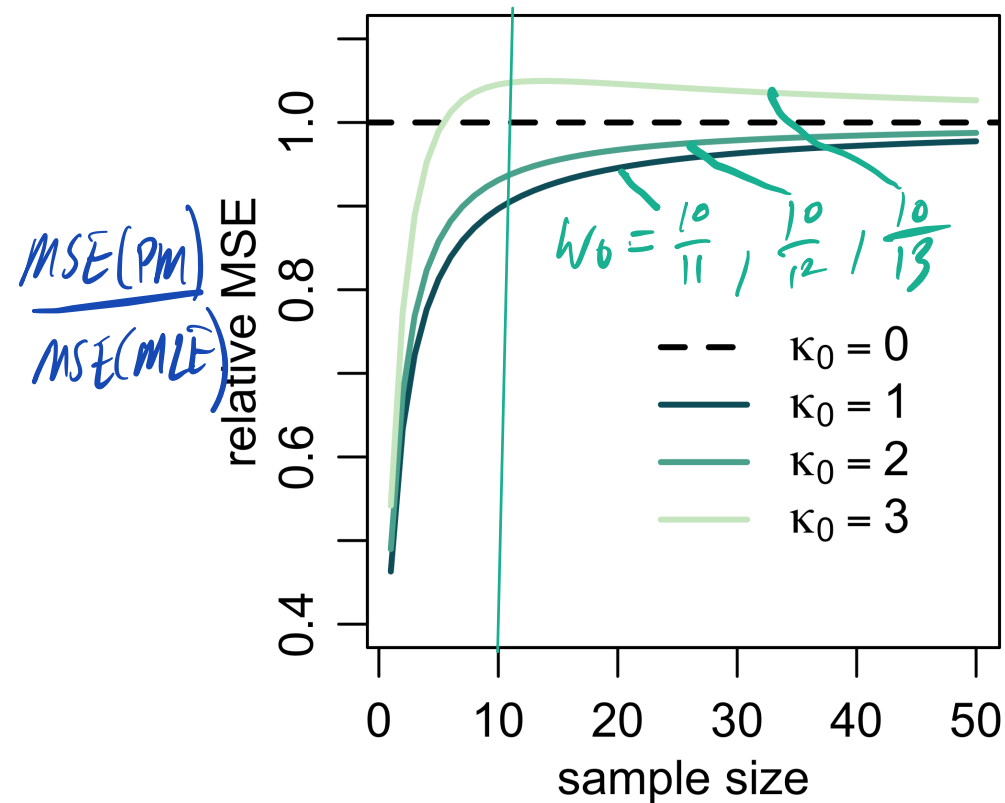
- Scoring on **IQ tests** is designed to yield a $N(100, 15)$ distribution for the general population
- We observe IQ scores for a sample of n individuals from a particular town and estimate μ , the town-specific IQ score
- If we lacked knowledge about the town, a natural choice would be $\mu_0 = 100$
- Suppose the true parameters for this town are $\mu = 112$ and $\sigma = 13$
 - The town is smarter on average than the general population

Example: IQ scores

- What is the mean squared error of the MLE? MSE of the posterior mean?
- $\text{MSE} [\hat{\mu}_{MLE}] = \text{Var} [\hat{\mu}_{MLE}] = \frac{\sigma^2}{n} = \frac{169}{n}$
- $\text{MSE} [\hat{\mu}_{PM} | \theta_0] = w^2 \frac{169}{n} + (1 - w)^2 144$
- Reminder: $w = \frac{n}{\kappa_0 + n}$. For what values of n and κ_0 is the MSE smaller for the posterior mean estimator than the maximum likelihood?

Example: IQ scores

$(\mu_0 - \mu) = 12$ (not knowable in practice)



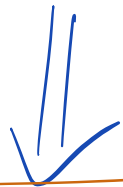
class: middle, center, inverse; background-image: none;

$$Y_1, \dots, Y_n \stackrel{\text{iid}}{\sim} N(\mu, \sigma^2)$$

known.

$$P(\mu) \sim N(\mu_0, \frac{\sigma^2}{k_0})$$

controls "confidence" of prior. Plays role of pseudo-sample size.



$$P(\mu | y_1, \dots, y_n, \sigma^2) \sim N(\mu_n, \sigma_n^2)$$

$$\mu_n = w \bar{y} + (1-w) \mu_0$$

$$\sigma_n^2 = w \frac{\sigma^2}{n}$$

$$w = \frac{n}{n + k_0}$$

Frequentist Property of Posterior Mean

$$MSE = Bias^2 + Var.$$

$$\begin{aligned} Var(\hat{\mu}_{pm}) &= Var(w\bar{Y} + (1-w)\mu_0) \\ &= w^2 Var(\bar{Y}) = w^2 \frac{\sigma^2}{n} \end{aligned}$$

$$Bias(\hat{\mu}_{pm}) = E[w\bar{Y} + (1-w)\mu_0] - \mu$$

$$= w\mu - (1-w)\mu_0 - \mu$$

$$= (1-w)(\mu_0 - \mu)$$

$$MSE: (1-w)^2 (\mu_0 - \mu)^2 + w^2 \frac{\sigma^2}{n}$$

$$MSE \text{ for MLE: } \frac{\sigma^2}{n}$$

Decision Theory

Why the posterior mean?

- Often times we need to make a “decision” by providing a single estimate
- The posterior provides a full distribution over θ , which can be summarized in infinitely many ways
- Specify a *loss function* which describes the cost of estimating $\hat{\theta}$ when the truth is θ

Bayes Estimators

- The *specific* loss function: $L(\hat{\theta}, \theta)$ (penalty of reporting $\hat{\theta}$ when truth is θ)
 - Squared error: $L(\hat{\theta}, \theta) = (\hat{\theta} - \theta)^2$
 - Absolute error: $L(\hat{\theta}, \theta) = |\hat{\theta} - \theta|$
- The Bayes risk is the posterior expected loss:

$$E_{\theta|y}[L(\hat{\theta}, \theta)] = \int \underbrace{L(\hat{\theta}, \theta)}_{\text{average}} p(\theta | y) d\theta$$

Belief.

- Choose an estimator of θ based on minimizing the Bayes risk.
- An estimator $\hat{\theta}$ is said to be a **Bayes estimator** if it minimizes the Bayes risk among all estimators.

Squared error loss

$$\min_{\hat{\theta}} E_{\theta|y}(\hat{\theta} - \theta)^2 = \min_{\hat{\theta}} \int (\hat{\theta} - \theta)^2 p(\theta | y) d\theta$$

Differentiate with respect to $\hat{\theta}$ and set equal to zero:

$$\frac{d}{d\hat{\theta}} \int (\hat{\theta} - \theta)^2 p(\theta | y) d\theta =$$

$$\int 2(\hat{\theta} - \theta) p(\theta | y) d\theta = 0$$

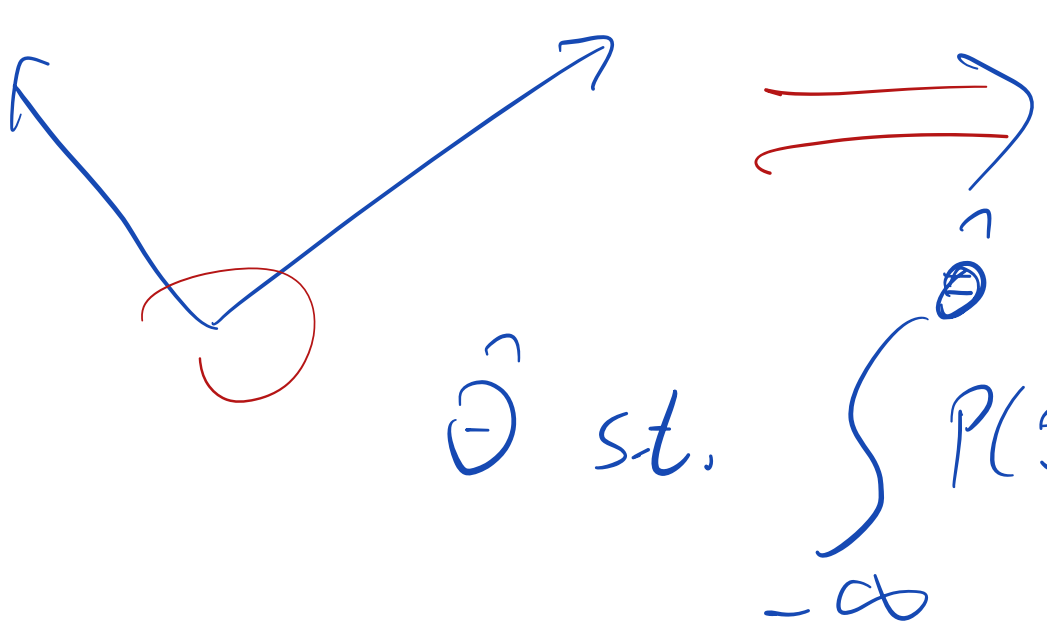
$$\cancel{2} \hat{\Theta} \underbrace{\int P(\theta/y) d\theta}_{\text{" "}} = \cancel{2} \int \theta P(\theta/y) d\theta = E[\theta/y] \text{ "post. Mean"}$$

$$\hat{\Theta} = E[\theta/y]$$

Absolute loss

$$\min_{\hat{\theta}} E_{\theta|y} |\hat{\theta} - \theta| = \min_{\hat{\theta}} \int |\hat{\theta} - \theta| p(\theta | y) d\theta$$

Differentiate with respect to $\hat{\theta}$ and set equal to zero:



$\hat{\theta} = \text{posterior Median}$

$\hat{\theta} \text{ st. } \int_{-\infty}^{\hat{\theta}} P(\theta|y) d\theta = 0.5$

What if σ^2 is not known?

$$Y_1, \dots, Y_n \sim N(\mu, \sigma^2)$$

$$\Rightarrow L(\mu, \sigma^2) \propto \left(\frac{1}{\sqrt{2\pi\sigma^2}}\right)^n e^{-\frac{(\bar{y} - \mu)^2}{2\sigma^2/n}}$$

$$P(\mu, \sigma^2) = \underbrace{P(\mu | \sigma^2)}_{N(\mu_0, \frac{\sigma^2}{K_0})} \underbrace{P(\sigma^2)}_{??}$$

2-parameter inference

$$\rightarrow P(\mu, \sigma^2 | y_1, \dots, y_n)$$

Joint inference for normal parameters

- In practice, both μ and σ are unknown in the normal model.
- We've considered models when either μ is unknown but σ is known.
- In practice neither is none!
- Need to specify a joint prior distribution: $p(\mu, \sigma)$
- This is our first look at *multi-parameter* models.

2 params

Joint inference for the mean and variance

In the normal model we typically factorize the prior distribution $p(\mu, \sigma) = p(\mu \mid \sigma)p(\sigma)$.

Specifically:

$$\sigma \propto \frac{1}{\sigma^2}$$

*Improper Prior.
Non-informative.
Reasonable Default.*

$$\mu \mid \sigma \sim \text{normal}(\mu_0, \sigma^2 / \kappa_0)$$

*just
like
before*

$$Y_1, \dots, Y_n \mid \mu, \sigma \sim \text{i.i.d. normal}(\mu, \sigma^2)$$

- μ_0 is interpreted as the prior sample mean
- κ_0 is a prior sample size

Pseudo-Data
Interpretation.

Example: midge wing length

- Modeling wing length of different species of midge (small, two-winged flies)
- From prior studies: mean wing length close to 1.9mm.
- Prior mean for μ is $\mu_0 = 1.9$ and non-informative prior for σ
-
- Prior sample sizes: choose $\kappa_0 = 1$ *Imagine one previous fly (weak) prior*
- $(\bar{y}, s^2) = (1.804, 0.0169)$ are the sufficient statistics

$P(\mu, \sigma^2 | y_1, \dots, y_n)$ via Computation

Working with the log posterior

- As always, we will write down $p(\theta \mid y) \propto p(y \mid \theta)p(\theta)$
- In code, we always work with the **log-posterior** for numerical reasons
 - Mathematically it makes no difference, but computationally it is important
 - $L(\theta) \propto \prod p(y_i \mid \theta)$ is very small for moderate sample size (underflow)
 - $\ell(\theta) = \sum \log(p(y_i \mid \theta))$ is numerically stable
- Monte Carlo methods only require that we can evaluate the log posterior

Grid approximation to the posterior distribution

Write a function that evaluates

```
1 log_normal_posterior <- Vectorize(function(mu, sigma) {  
2   ## log likelihood  
3   sum(dnorm(y, mu, sigma, log=TRUE)) +  
4   ## plus log prior  
5   dnorm(mu, mu0, sigma/sqrt(k0), log=TRUE) +  
6   -2*log(sigma)  
7 } )  
8  
9  
10  
11 post_grid <- as_tibble(  
12   expand.grid(seq(1.6, 2.0, by=0.001),  
13               seq(0.01, 0.25, by=0.001)))  
14 colnames(post_grid) <- c("mu", "s")
```

$\log P(\mu, \sigma^2 | y_1, \dots, y_n)$
 $= \log L(\mu, \sigma^2) + \log P(\mu, \sigma^2)$
 $\log P(\mu | \sigma^2)$
 $\log P(y | \mu, \sigma^2)$
Grid of Values

Why log Posterior?

Post. $\left(\prod_i \underbrace{P(y_i | \theta)} \right) P(\theta)$

↑ these are ^{very} small numbers. Leads to underflow in the computer.

log Post:

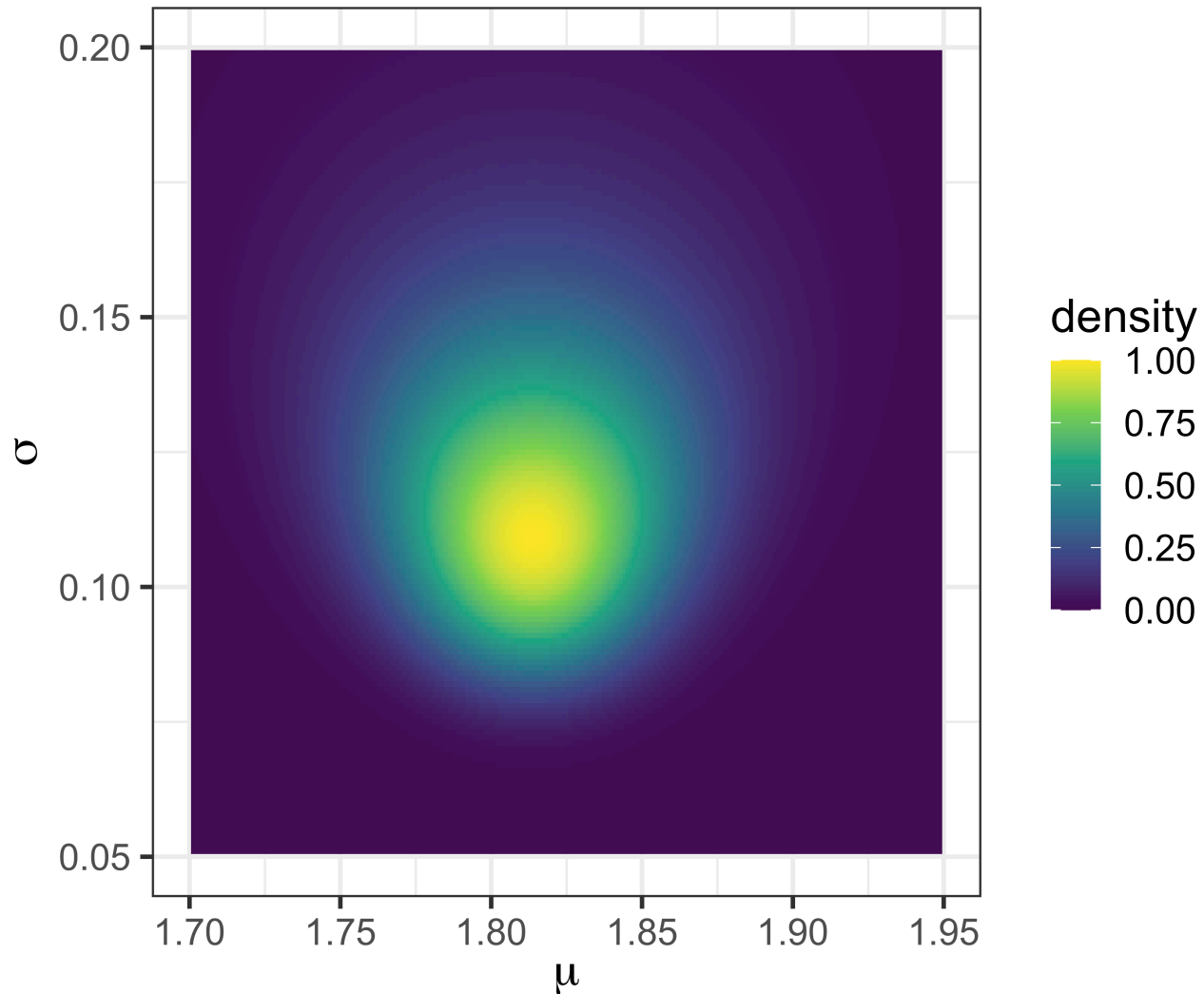
$$\sum \log P(y_i | \theta) + \log P(\theta)$$

Stable

Grid approximation to the posterior distribution

```
1 post_grid %>%  
2   mutate(log_density = log_normal_posterior(mu, s)) %>%  
3   mutate(density = exp(log_density - max(log_density))) %>%  
4   ggplot() +  
5   geom_raster(aes(mu, s, fill=exp(log_density))) + #<<  
6   xlim(c(1.7, 1.95)) + ylim(c(0.05, 0.2)) +  
7   xlab(expression(mu)) +  
8   ylab(expression(sigma)) +  
9   theme_bw() +  
10  scale_fill_continuous(type="viridis")
```

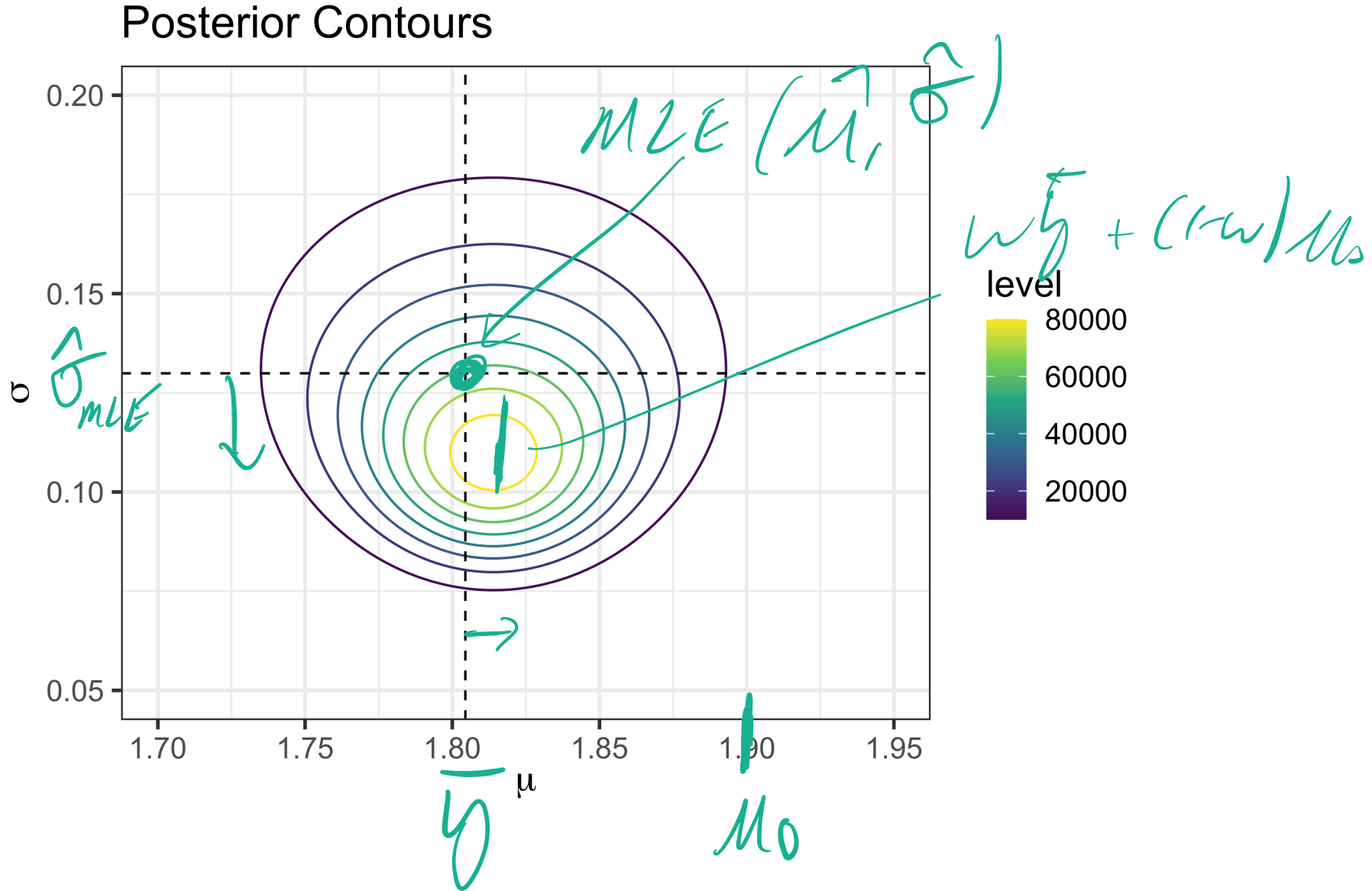
Grid approximation



Contour Plot

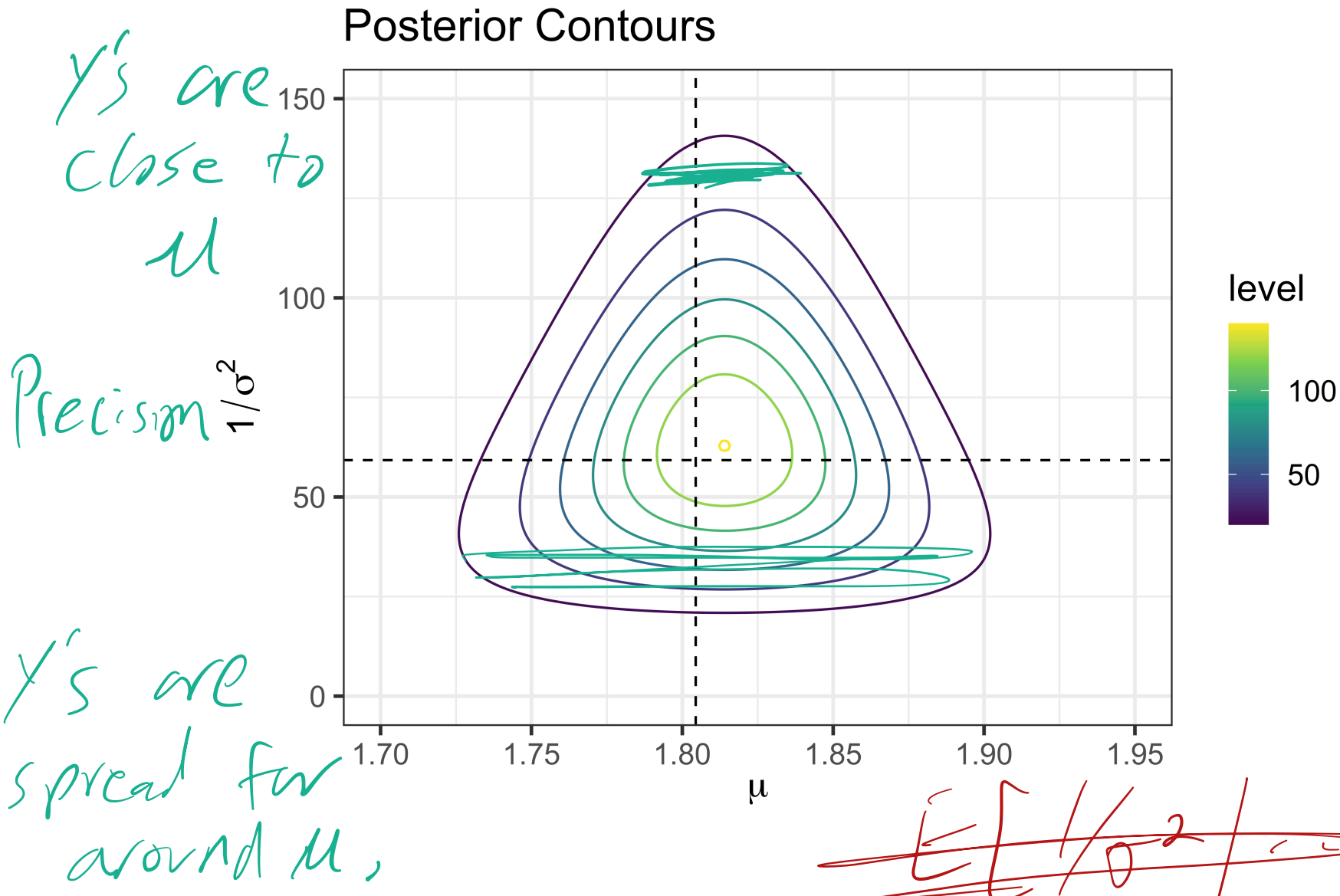
```
1 post_grid %>%
2   mutate(density = exp(log_density - max(log_density))) %>%
3   ggplot() +
4   geom_contour(aes(mu, s, z=density, colour=stat(level)), size=2) + #<<
5   xlim(c(1.7, 1.95)) + ylim(c(0.05, 0.2)) +
6   xlab(expression(mu)) + ylab(expression(sigma)) +
7   ggtitle("Posterior Contours") +
8   theme_bw(base_size=16) +
9   scale_color_continuous(type="viridis")
```

Contour Plot (Standard Deviation)



Contour Plot (Precision)

$$P(\mu, 1/\sigma^2 | y_{1:n}, y_n)$$



Sampling from the joint posterior

- Contour and raster plots allow us to visualize the posterior (in two dimensions)
 - Need to know approximately where the high posterior density is (not easy)
- When we have more than 2 parameters visualization isn't feasible
- How do we summarize the posterior?
 - e.g. posterior means, posterior probabilities, intervals, etc..

Sampling from the joint posterior

- One approach: convert multi-parameter distribution into the product of many one parameter distributions.

Markov Chain Monte Carlo

- Markov Chain Monte Carlo (MCMC)
- More effective approach to sampling from multi-parameter distributions
- Samples in MCMC are **not** independent samples

Random
Sampling
approx integral

Dependence

MCMC Sampling with Stan

Stan

"Probabilistic

Programming
Languages"



Others

- NumPyro

- PyMC

"A state-of-the-art platform for statistical modeling and high-performance statistical computation."

<http://mc-stan.org/>

Monte Carlo Sampling with Stan

- Stan is a “probabilistic programming language” with its own simple syntax
- R interface to Stan via `cmdstanr` package
- Define parameters θ and datatypes y
- Provide sampling model $p(y \mid \theta)$ and prior $p(\theta)$
- Stan translates model syntax into the log posterior density $\log(p(\theta \mid y))$ and automatically generates samples from this density

Example Stan File

```
1 // The input data is a vector 'y' of length 'N'.
2 data { #<<
3   int<lower=0> N;
4   real<lower=0> k0;
5   vector[N] y;
6 }
7
8 // The parameters accepted by the model. Our model
9 // accepts two parameters 'mu' and 'sigma'.
10 parameters { #<<
11   real mu;
12   real<lower=0> sigma;
13 }
14
15 // The model to be estimated. We model the output
16 // 'y' to be normally distributed with mean 'mu'
17 // and standard deviation 'sigma'.
18 model { #<<
```

The Stan File

- A stan file ends in `.stan`
- Three important program blocks in a stan file:
 - Data block
 - Parameter block
 - Model block
 - Each block encapsulated in brackets, `{ ... }`.
- Stan needs data types:
 - `int`: integer valued data
 - `real`: continuous data or parameters
- Need to end every line with a semi-colon!
- Need to compile the Stan program before running.

Defining the input data in Stan

```
1 // The input data is a vector 'y' of length 'N'.  
2 data {  
3   int<lower=0> N;  
4   real<lower=0> k0;  
5   vector[N] y;  
6 }
```

Defining the model parameters in Stan

```
1 // The parameters accepted by the model. Our model
2 // accepts two parameters 'mu' and 'sigma'.
3 parameters {
4   real mu;
5   real<lower=0> sigma;
6 }
```

Defining the model in Stan

```
1 // The model to be estimated. We model the output
2 // 'y' to be normally distributed with mean 'mu'
3 // and standard deviation 'sigma'.
4 model {
5   target += -2*log(sigma); //sigma prior
6   mu ~ normal(1.9, sigma^2/k0); // mu prior
7   y ~ normal(mu, sigma); // data model
8 }
```

Running MCMC with 4 sequential chains...

Chain 1 finished in 0.0 seconds.

Chain 2 finished in 0.0 seconds.

Chain 3 finished in 0.0 seconds.

Chain 4 finished in 0.0 seconds.

All 4 chains finished successfully.

Mean chain execution time: 0.0 seconds.

Total execution time: 0.5 seconds.

```
1 ## Load the rstan library
2 library(cmdstanr)
3
4 # compile stan model. This may take a minute. #<<<
```

```
5 stan_model <- cmdstan_model(stan_file="normal_model.stan") #<<
6
7 ## data is a list, arguments must match the
8 ## arguments in data block of the stan file
9 stan_fit <- stan_model$sample(data=list(N=n, y=y, k0=0.1),
10                               refresh=0)
```

Stan: The Basics

```
1 library(cmdstanr)
2
3 # compile stan model. This may take a minute. #<<
4 stan_model <- cmdstan_model(stan_file="normal_model.stan")
5
6 ## data is a list, arguments must match the
7 ## arguments in data block of the stan file
8 stan_fit <- stan_model$sample(data=list(N=n, y=y, k0=0.1),
9                               refresh=0) #<<
```

Running MCMC with 4 sequential chains...

Chain 1 finished in 0.0 seconds.

Chain 2 finished in 0.0 seconds.

Chain 3 finished in 0.0 seconds.

Chain 4 finished in 0.0 seconds.

All 4 chains finished successfully.

Mean chain execution time: 0.0 seconds.

Total execution time: 0.5 seconds.

Stan: The Basics

```
1 ## Convert Stan output to a list of MC samples
2 samples <- stan_fit$draws(format="df")
3 samples
```

```
# A draws_df: 1000 iterations, 4 chains, and 3 variables
```

```
  lp__    mu sigma
1    17  1.8 0.137
2    17  1.8 0.130
3    16  1.8 0.096
4    16  1.7 0.140
5    16  1.9 0.140
6    17  1.8 0.148
7    17  1.9 0.134
8    16  1.8 0.183
9    15  1.7 0.133
10   16  1.8 0.173
```

```
# ... with 3990 more draws
```

```
# ... hidden reserved variables {'.chain', '.iteration', '.draw'}
```

Stan: The Basics

```
1 ## names in list are the defined parameters
2
3 mu_samples <- samples$mu
4 sigma_samples <- samples$sigma
5
6 ## By default there will be 4000 samples
7 length(mu_samples)
```

```
[1] 4000
```


Stan: The Basics

```
1 ## Information about MC samples
2 summary(mu_samples)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
1.618	1.778	1.804	1.805	1.832	2.053

```
1 summary(sigma_samples)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
0.06176	0.10335	0.12029	0.12562	0.14134	0.36295

```
1 summary(1/sigma_samples^2)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
7.591	50.060	69.110	74.864	93.620	262.149

Stan: The Basics

Shortcut for summary statistics:

```
1 ## Information about MC samples
2 stan_fit$summary()
```

A tibble: 3 × 10

	variable	mean	median	sd	mad	q5	q95	rhat	ess_bulk	ess_tail
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	lp__	16.4	16.7	1.09	0.787	14.3	17.5	1.00	1429.	1578.
2	mu	1.81	1.80	0.0436	0.0404	1.74	1.87	1.00	2243.	1557.
3	sigma	0.126	0.120	0.0320	0.0275	0.0846	0.186	1.00	1793.	1851.

Visualizing Posterior Samples from Stan

```
1 tibble(Mean = mu_samples, Precision=1/sigma_samples^2) %>%  
2   ggplot() +  
3   geom_point(aes(x=Mean, y=Precision)) +  
4   theme_bw(base_size=16)
```

